

Week 5: The Avellaneda-Stoikov Engine

Implementation, calibration, and backtest – Milestone 1

North Carolina State University | Summer 2026

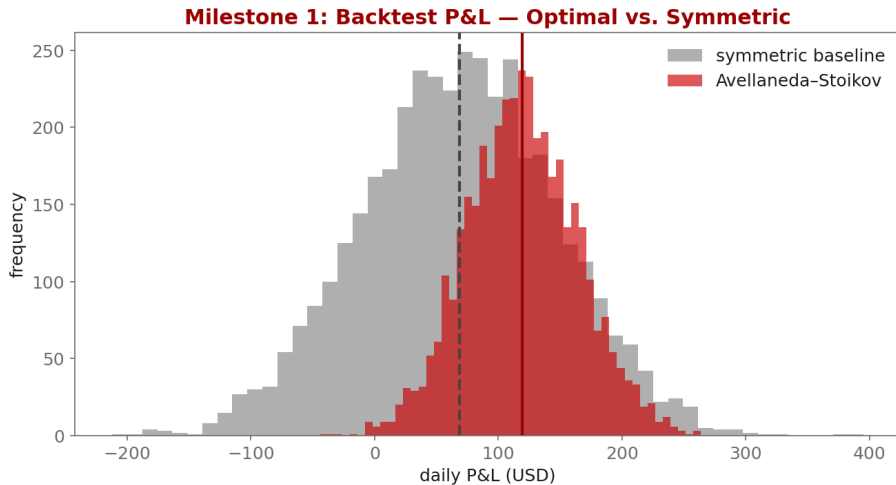
NC STATE UNIVERSITY

Industry Mentor: Dendi Suhubdy (Bitwyre) | Guest: Fenni Kang (Coincall)

- ▶ Implement the quotes as a tested PyTorch engine
- ▶ Calibrate the fill intensities by maximum likelihood
- ▶ Backtest on one month of Coincall BTC-PERP data

- ▶ Engine architecture; state from the L2 book
- ▶ Map continuous-time quantities to discrete events
- ▶ Calibrate $\lambda(\delta) = A \exp(-\kappa \delta)$ from fills

- ▶ Maximum-likelihood fit of A and κ with goodness-of-fit
- ▶ Replay engine + fill simulator + P&L attribution
- ▶ Reproducibility under a fixed seed



$$\mathcal{L}(A, \kappa) = \prod_i \lambda(\delta_i) e^{-\lambda(\delta_i) \Delta t_i}$$

Why: Fills are a (doubly stochastic) Poisson process, so the likelihood of the observed fills given quote distances δ_i is a product of Poisson densities. We fit by MAXIMUM LIKELIHOOD rather than least-squares because the data are event counts, not Gaussian residuals.

$$-\ln \mathcal{L} = \sum_i [\lambda(\delta_i) \Delta t_i - \ln \lambda(\delta_i)]$$

Why: Minimizing this negative log-likelihood is convex in $\ln A$ and well-behaved in κ ; PyTorch autograd gives the gradients, so a few hundred Adam steps converge.

- ▶ Build the intensity-calibration set from trades vs. your quotes:
 - ▶ Trades: GET /open/option/trade/lasttrade/v1/{symbol} (or trade history)
 - ▶ Book: GET /open/futures/order/orderbook/v1/{symbol} for the mid
- ▶ Offline: `coincall_btc_perp_trades_2025Q4.parquet + ..._l2_2025Q4.parquet`
- ▶ A pre-aggregated `coincall_intensity_calibration_set.parquet` is provided
- ▶ $\text{delta}_i = |\text{fill price} - \text{mid at fill}|$; group fills by delta bucket for the MLE

Cross-check exact paths at <https://docs.coincall.com/>


```
import torch
# delta: quote distances at which fills occurred; dt: exposure time per fill
delta = torch.tensor([...]); dt = torch.tensor([...])
logA = torch.zeros(1, requires_grad=True)
kappa = torch.ones(1, requires_grad=True)
opt = torch.optim.Adam([logA, kappa], lr=0.05)
for step in range(2000):
    lam = torch.exp(logA) * torch.exp(-kappa.clamp(min=1e-3) * delta)
    nll = (lam * dt - torch.log(lam)).sum() # Poisson negative log-lik
    opt.zero_grad(); nll.backward(); opt.step()
print("A =", torch.exp(logA).item(), " kappa =", kappa.item())
```

- ▶ Avellaneda-Stoikov (2008) [implementation]; Gueant (2016), Ch. 4